

Introduction

The goal of this report is to highlight the benefits of using SQL queries, views, constraints, and Entity-Relationship Diagrams to better understand a bank's data on its clients. These tools can help the bank to extract, organise, and analyse this data, leading to improved insights and services.

SQL queries allow the bank to select specific pieces of information from its data, enabling it to answer specific questions and make data-driven decisions. For example, the bank could use a SQL query to retrieve all account transactions for a particular customer, or to identify all clients with a certain credit score. Views and constraints can be used to structure and organise the data in a way that is more intuitive and easier to work with. For instance, the bank could create a view that combines data from multiple tables, such as account transactions and credit card usage, to provide a more comprehensive overview of a customer's financial activity.

By utilising these SQL tools and techniques, the bank can more effectively use its data to gain insights into its clients and improve its services. For example, the bank could use the results of a SQL query to identify customers who may be in need of additional financial products or services, or to identify potential risks in its loan portfolio. By organising and structuring the data in a more intuitive way, the bank can make it easier for its employees to work with and analyse the data, leading to more efficient and effective decision-making. And by visualising the relationships between different data entities, the bank can more easily understand the connections between different pieces of information, and use this understanding to improve its services.

Data Creation

The dataset is the [PKDD'99 financial dataset](#) formulated to help banks assess the likely outcome of their loans. It comprises eight related tables and a million total rows with information about a range of features relating to the bank's clients & their interactions with the bank. I chose this dataset in order to look at other ways the bank can use the data they have collected to help meaningfully inform their service and understand their clients.

Regarding implementation, the data set is made up of a set of eight CSV's that were imported into the notebook. The first obvious issue came as a result of the date formatting. The values were simply integers containing the date in a YYYYMMDD format. To resolve this three functions were created to transform the integer into a datetime object that were applied to the tables before they were converted into an SQL table with a datetime column restriction.

The database tables are created empty in order to ensure all constraints on the data are established, foreign key relations are properly defined and each table has a proper primary key. The method of creation allows me to specify the allowed data type for each column and also establishes non null constraints unless otherwise specified. Additional constraints are also included for categorical data to make sure erroneous categories aren't accidentally added to the data. The construction is validated by inspecting the db schema to confirm it is properly constructed.

I also decided to remove 3 columns from the transaction database as I deemed them incomplete and superfluous to requirement. I also made sure to check the tables have been properly constructed, verifying the primary and foreign keys for completeness.

Database technology

The project runs on Jupyter Notebook to demonstrate a cohesive report on the implementation of the solution. In order to pre-process and clean the data I largely used python to create functions for cleaning data and making use of pandas to read in the data from the CSV files as well as to initially inspect the data to inform the construction of the SQL tables.

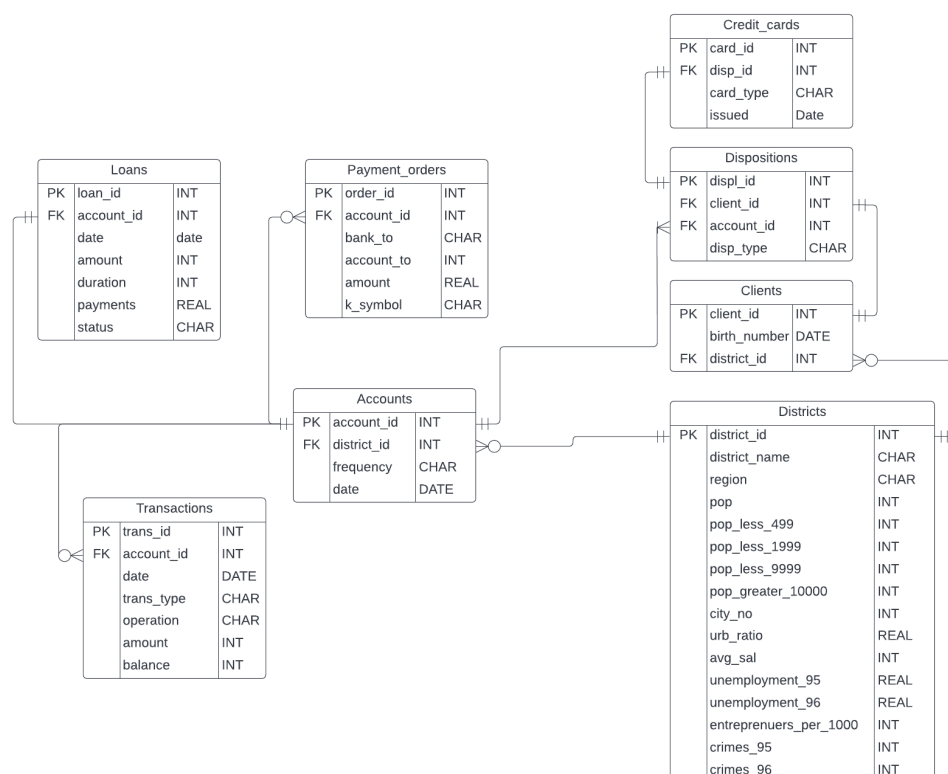
Creation of those tables was achieved through an OOP method of creating classes for each table to specify data types, relationships & and constraints on the tables. Due to a bug with a pandas to_sql function I used an iteration on the pandas tables and SQL insert commands to fill the database after construction. Further use of SQL via sqlite3 was used in order to set up queries and sometimes a combination of peewee and sqlite3 in order to facilitate easily readable responses from the queries or to set up View tables. DB browser is used to verify the functionality of the relational construction and visually check the tables and views are being made as expected

Additional libraries include OS for file management, datetime for datetime object construction.

Data modelling

Entity-Relationship Diagrams provide a visual representation of the relationships between different data entities, such as clients, accounts, loans, and credit cards. These diagrams can be used to understand the connections between these entities, and to identify patterns and trends in the data. For example, the bank could use an Entity-Relationship Diagram to see which clients have multiple accounts, or to understand the distribution of credit card usage among its customers.

ER/EER Diagram



Description of the Entities

- Entity **account** (4500 objects in the file ACCOUNT.CSV) - each record describes static characteristics of an account, with the following attributes: (unique) **account_id**, **district_id**, **frequency** (frequency of issuance of statements), **date** (date of creation of the account).
- Entity **client** (5369 objects in the file CLIENT.CSV) - each record describes characteristics of a client that can manipulate accounts, with the following attributes: (unique) **client_id**, **birth_number**, **district_id**.
- Entity **disposition** (5369 objects in the file DISP.CSV) - each record relates together a client with an account i.e. this relation describes the rights of clients to operate accounts, with the following attributes a unique **disp_id**, **client_id**, **account_id**, and **type of disposition** (owner/user).
- Entity **permanent order** (6471 objects in the file ORDER.CSV) - each record describes characteristics of a payment order, with the following attributes: **order_id**, **account_id**, **bank_to** (bank of the recipient), **account_to** (account of the recipient), **amount**, **K-symbol** (characterisation of the payment).
- Entity **transaction** (1056320 objects in the file TRANS.CSV) - each record describes one transaction on an account, with the following attributes: **unique trans_id**, **account_id**, **date**, **type (+/- transaction)**, **operation (mode of transaction)**, **amount**, **balance (balance after transaction)**, **k_symbol (characterization of the transaction)**, **bank**, **account**.
- Entity **loan** (682 objects in the file LOAN.CSV) - each record describes a loan granted for a given account, with the following attributes: **unique loan_id**, **account_id**, **date**, **amount**, **duration**, **payments**, **status** (either A,B,C,or D).
- Entity **credit card** (892 objects in the file CARD.CSV) - each record describes a credit card issued to an account, with the following attributes: **a unique card_id**, **disp_id**, **type (possible values are "junior", "classic", "gold")**, and the date issued.
- Both Entities "loan" and "credit card" describe some services which the bank offers to its clients.
- Entity **demographic data** (77 objects in the file DISTRICT.CSV) - each record describes demographic characteristics of a district that are publically available. These are the following attributes: **district name**, **region**, **no. of inhabitants**, **no. of municipalities with inhabitants < 499**, **no. of municipalities with inhabitants 500-1999**, **no. of municipalities with inhabitants 2000-9999**, **no. of municipalities with inhabitants >10000**, **no. of cities**, **ratio of urban inhabitants**, **average salary**, **unemployment rate '95**, **unemployment rate '96**, **no. of entrepreneurs per 1000 inhabitants**, **no. of committed crimes '95**, **no. of committed crimes '96**

Relationships between entities and their attributes

- An **account** needs at least one **client** (owner) but can also have many clients (other disponents) and therefore at least one disposition that connects a unique client with a unique card. The **dispositions** are then linked to one and only one account, however, an account can have one or many dispositions.
- To be a **client** you need to be registered for at least one disposition (does not have to be as an owner), but a client can also have multiple dispositions (1 or many).
- Each **disposition** is unique to a **card** and a **client**, however, a client and an and a card both can have at least one or multiple dispositions. The disposition, however, can only match 1 client and one card at a time. (one-to-one) and (one-or-many).
- An order is issued from one and only one account, however an account can issue zero to many orders.
- At most one **loan** can be granted for an **account**. And a loan needs one and only one account.
- An **account** can make zero to many **transactions**, however, a unique transaction can only be made by one and only one account.
- Both an account and a client are registered in one and only one **district** (they don't have to be the same). A district can have zero to many accounts registered.

Database usage

In this section I will be demonstrating how the database user can utilise this database in an analysing way in order to understand their business better by executing numerous queries.

Query 1: List the districts in descending order according to how many clients they have

- **Purpose:** With this query I can list all the districts in a descending order from the 'Districts' table according to the number of clients they have. This is useful for the bank to understand where their clients are based which could for example help them decide to which branches send more staff to provide any services.
- **Input parameters and conditions:** There are no input parameters or conditions.
- **Expected outputs:** In this query, the expected outputs are the column 'district_id', the count of 'district_id' from the 'Clients' table and 'district_name', 'region' from the 'Districts' table. The expected output is ordered in a descending order by the count of 'district_id':

(1, 'Hl.m. Praha', 'Prague', 663)
 (74, 'Ostrava - mesto', 'north Moravia', 180)
 (70, 'Karvina', 'north Moravia', 169)
 (54, 'Brno - mesto', 'south Moravia', 155)
 (64, 'Zlin', 'south Moravia', 109)
 (72, 'Olomouc', 'north Moravia', 104)
 (68, 'Frydek - Mistek', 'north Moravia', 86)
 (46, 'Nachod', 'east Bohemia', 76)
 (52, 'Usti nad Orlici', 'east Bohemia', 73)
 (5, 'Kolin', 'central Bohemia', 71)
 (8, 'Mlada Boleslav', 'central Bohemia', 69)
 (33, 'Decin', 'north Bohemia', 69)
 ...

Query 2: Count how many different credit cards there are

- **Purpose:** This query lets the database user know how the number of credit card users distributes between different types of credit cards. This could be helpful, for example, for the bank to understand which credit card types are more popular which is useful in developing new credit card products or marketing strategies.
- **Input parameters and conditions:** There are no input parameters or conditions.
- **Expected outputs:** The expected output includes the 'type' column, the 'Credit_cards' table and the count for that column. The results are order in a descending order by the count for different types of credit cards:

('classic', 659)
 ('junior', 145)
 ('gold', 88)

Query 3: List the average salary and average yearly (amount) of a loan for each district

- **Purpose:** This query gives the bank a good comparison of each district's average salary and average yearly amount of loan. The bank can use this information on decisions whether to grant on a loan application and to ensure that granted loans are sustainable and successful as

comparing the average salary in a district with the average yearly loan amount, gives a better sense of the borrower's financial situation and any potential risks when lending to them.

- **Input parameters and conditions:** There are no input parameters or conditions.
- **Expected outputs:** Expected output includes the 'district_id' column from the 'Accounts' table which previously has been LEFT JOINED with the 'Loans' table, 'avg_sal' column from 'Districts' table and the average yearly loan for a relevant district:

(1, 12541, 52067.42857142857)

(8, 11277, 50124.0)

(26, 10787, 32608.0)

(74, 10673, 58908.0)

(39, 10446, 40178.4)

(70, 10177, 54083.0)

(10, 10124, 53210.4)

(14, 10045, 64285.5)

(7, 9920, 53202.0)

(54, 9897, 49003.5)

(68, 9893, 44850.75)

(41, 9832, 51064.0)

(4, 9753, 42958.28571428572)

(34, 9675, 47393.333333333336)

(30, 9650, 61230.0)

(64, 9624, 54037.41176470588)

(11, 9622, 51855.27272727273)

...

Query 4: List the districts of those clients who are in dept or haven't paid their loan and see the demographic data of these districts

- **Purpose:** This query selects all the districts of those clients who are in dept or haven't paid their loan and the demographic data of them. This is useful for deciding on approval of any loan applications as it helps to identify any districts that are at higher risk of default.
- **Input parameters and conditions:** Database user can specify for which 'status' they want to see the data. For this specific query 'B' and 'D' are used.

- **Expected outputs:** From the 'Loans' table: 'account_id', 'B' and 'D' values from 'status', 'district_id' and all the columns from the 'Districts' table:

(19, 'B', 21, 'Tabor', 'south Bohemia', 103347, 87, 16, 7, 1, 7, 67, 9104, 1.51, 2.07, 123, 2299, 2354)
 (37, 'D', 20, 'Strakonice', 'south Bohemia', 70646, 94, 14, 3, 1, 4, 58.4, 8547, 2.65, 3.64, 120, 1563, 1542)
 (103, 'D', 44, 'Chrudim', 'east Bohemia', 105606, 77, 26, 7, 2, 7, 53, 8254, 2.79, 3.76, 97, 2166, 2325)
 (347, 'B', 65, 'Znojmo', 'south Moravia', 114200, 101, 41, 4, 1, 4, 43.7, 8403, 5.74, 5.72, 105, 2157, 2718)
 (426, 'D', 1, 'Hl.m. Praha', 'Prague', 1204953, 0, 0, 0, 1, 1, 100, 12541, 0.29, 0.43, 167, 85677, 99107)

...

Query 5: Count how many transactions where made from 1993 to 1998

- **Purpose:** This query gives the database user an overview how many transactions were made from 1993 to 1998. This could be useful to track and analyse the bank's growth by identifying new trends in the volume of transactions.
- **Input parameters and conditions:** There are no input parameters or conditions.
- **Expected outputs:** Year which is extracted from the 'date' column from the Transactions table and the count for that value:

('1993', 28205)
 ('1994', 91628)
 ('1995', 133022)
 ('1996', 196779)
 ('1997', 284409)
 ('1998', 322277)

Dynamic transaction query

- **Purpose:** This query helps to manage a database transaction. More specifically, this query inserts a new loan application into the database, updates the status of the loan application, and then deletes any loan applications whose duration is less than 6 months. This query uses the BEGIN TRANSACTION, SAVE TRANSACTION, ROLLBACK TRANSACTION, and COMMIT TRANSACTION statements to manage the transaction and ensure that the database remains in a consistent state.
- **Input parameters and conditions:** There are no input parameters or conditions.
- **Expected outputs:** No expected outputs.

View 1: The total amount of loans each district has assumed

- **Purpose:** This view displays the total amount of debt each district is in, allowing the bank to further query this data to assist in the categorization of their clients and risk management. This can allow regional analysis and give the banks a good overview of all their total loans out.
- **Input parameters and conditions:** No input parameters or conditions.
- **Expected outputs:** district_name view created & displayed.

('Benesov', 887952),
('Beroun', 1460796),
('Blansko', 1191024),
('Breclav', 1176792),
('Brno - mesto', 4049400),
('Brno - venkov', 1232232),
('Bruntal', 1277796),
('Ceska Lipa', 462684),
('Ceske Budejovice', 2010924),

View 2: The total amount of loans each district has assumed

- **Purpose:** This simple but useful view allows banks to easily see how many of each type of card they have active allowing them to understand what their clientele is more inclined towards.
- **Input parameters and conditions:** No input parameters or conditions.
- **Expected outputs:** card_type view created and displayed.

[('classic', 659), ('gold', 88), ('junior', 145)]

View 3: The value of each districts total transactions

- **Purpose:** Similar to the previous view, this allows the bank to identify profitable districts that make frequent transactions, used in conjunction with this view the bank can create risk portfolios to assess districts that create frequent and valuable transactions against how much they may be in debt to the bank.

- **Input parameters and conditions:** No input parameters or conditions.
- **Expected outputs:** dis_trans view created in db & displayed.

(1, 5156492016.799973),
 (70, 1400877923.3000073),
 (74, 1350076137.9999857),
 (54, 1046367366.8999981),
 (64, 891400073.899996),
 (68, 855400655.6999986),
 (72, 835893440.2999935),
 (52, 649505697.1000011)

View 4: The number of accounts each district has and where they're located

- **Purpose:** Identifies regions with the most active accounts. This can help banks identify possible fraud if used in conjunction with regional population data to identify discrepancies as well as honing in on more lucrative regions that interact more with the bank identifying regions the bank may want to pay special attention to or showing areas the bank can expand into for growth.
- **Input parameters and conditions:** No input parameters or conditions.
- **Expected outputs:** dis_accounts view created in db and displayed.

[('Benesov', 'central Bohemia', 42),
 ('Beroun', 'central Bohemia', 50),
 ('Blansko', 'south Moravia', 50),
 ('Breclav', 'south Moravia', 44),
 ('Brno - mesto', 'south Moravia', 128)]

Log Trigger

- **Purpose:** This trigger keeps a log of any updates in the 'Loans' table by inserting the value that was changed, the value that was changed to and the timestamp of when the update was done. This trigger is helpful for managing the database.
- **Input parameters and conditions:** There are no input parameters or conditions.
- **Expected outputs:** There are no expected outputs.

Conclusion

In conclusion, the use of SQL queries, views, constraints, and Entity-Relationship Diagrams has proven to be a valuable tool in helping the bank to better understand and make use of its data on its clients. By leveraging the flexibility of coding in Python with the structural benefits of SQL, I was able to effectively clean and organise the data, and implement triggers and logs to keep track of updates and changes to the accounts.

The importance of data cleaning and organisation cannot be overstated, as it is essential for ensuring the accuracy and reliability of the data. By implementing triggers and logs, the bank can ensure the preservation of its data, and have a clear record of any changes that occur.

In the future, it may be interesting to explore the use of machine learning techniques to classify good loan applicants from bad ones. While SQL and simple querying may not be sufficient for this task, the data cleaning, structure, and observability that I have created in this report should provide a strong foundation for further investigation. By carefully considering the needs of the bank and its clients, and using a combination of SQL tools and machine learning techniques, further investigation on how to improve the services offered by the bank and better understand its customers would be an interesting research topic.